

**CS 524 – Introduction to Cloud Computing**  
**Final Project**

**NAME:** Aisiri Mandya Rajashekar

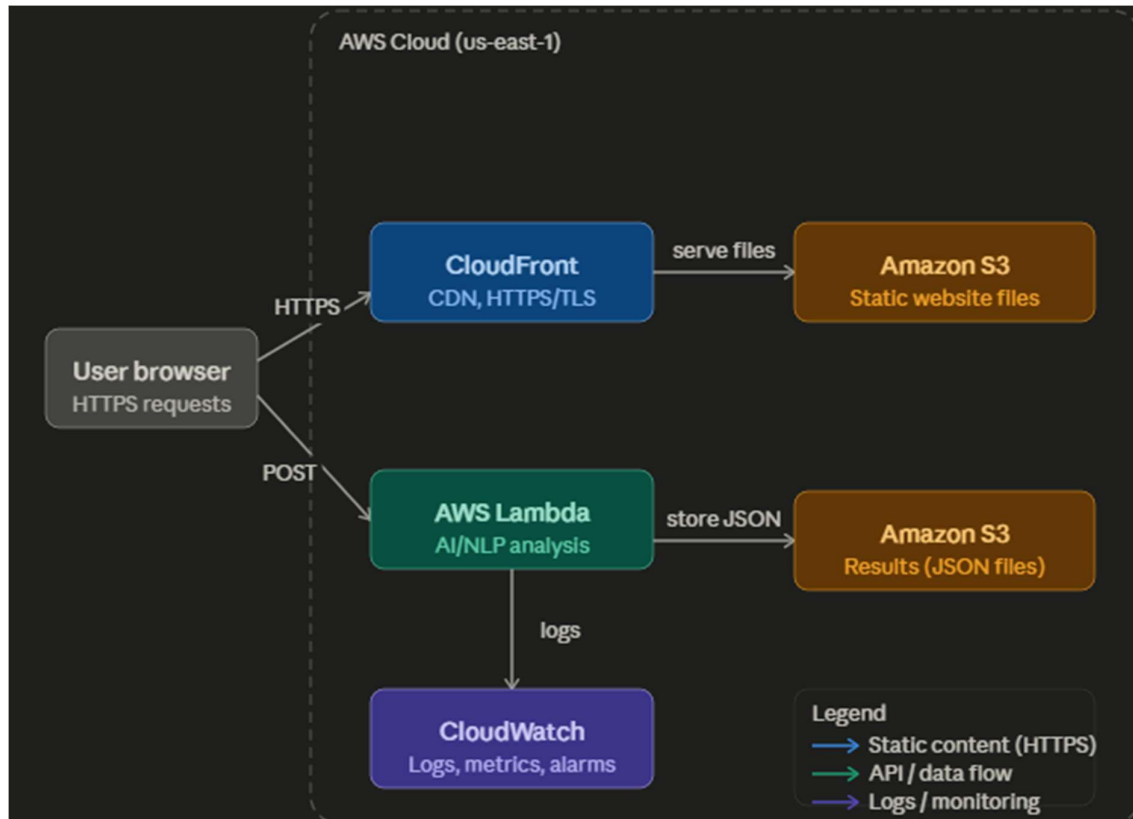
**E-mail:** arajashe@stevens.edu

**CWID:** 20043750

**AI-POWERED SECURE WEB**  
**APPLICATION**  
**Building an Intelligent Static Website on AWS**

## Phase 1- Architecture Diagram & Cost Estimate

### Step 1: Architecture diagram



This setup lets users access a website through CloudFront, send their input to a Lambda function for processing, and store the results in S3 while tracking activity in CloudWatch.

### Step 2: AWS Pricing Calculator

The screenshot shows the AWS Pricing Calculator interface. The 'My Estimate' section displays a total 12-month cost of 0.00 USD. The 'Getting Started with AWS' section includes buttons for 'Get started for free' and 'Contact Sales'. The 'My Estimate' table lists the services and their associated costs.

| Service Name        | Status | Upfront cost | Monthly cost | Description | Region                | Config Summary                                                                                                                                                                                         |
|---------------------|--------|--------------|--------------|-------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Amazon Simple St... | -      | 0.00 USD     | 0.00 USD     | -           | US East (N. Virginia) | S3 Standard storage (0.1 GB per month), PUT, COPY, POST, LIST requests to S3 Standard (100), GET, SELECT, and all other requests from S3 Standard (1000)                                               |
| AWS Lambda          | -      | 0.00 USD     | 0.00 USD     | -           | US East (N. Virginia) | Architecture (x86), Architecture (x86), Invoke Mode (Buffered), Amount of ephemeral storage allocated (512 MB), Number of requests (100000 per month)                                                  |
| Amazon CloudFront   | -      | 0.00 USD     | 0.00 USD     | -           | US East (N. Virginia) | Data transfer out to internet (0.001 GB per month), Number of requests (HTTPS) (100 per month)                                                                                                         |
| Amazon CloudWatch   | -      | 0.00 USD     | 0.00 USD     | -           | US East (N. Virginia) | Number of metrics (100), Number of metrics (100), Number of metrics (100), Number of metrics (100), Number of metrics (100), Number of metrics (100), Number of metrics (100), Number of metrics (100) |

I opened the AWS Pricing Calculator and added S3, Lambda, CloudFront, and CloudWatch with low usage values. The total came out to \$0.00 which confirmed everything fits within the free tier.

### Step 3: Design Explanation

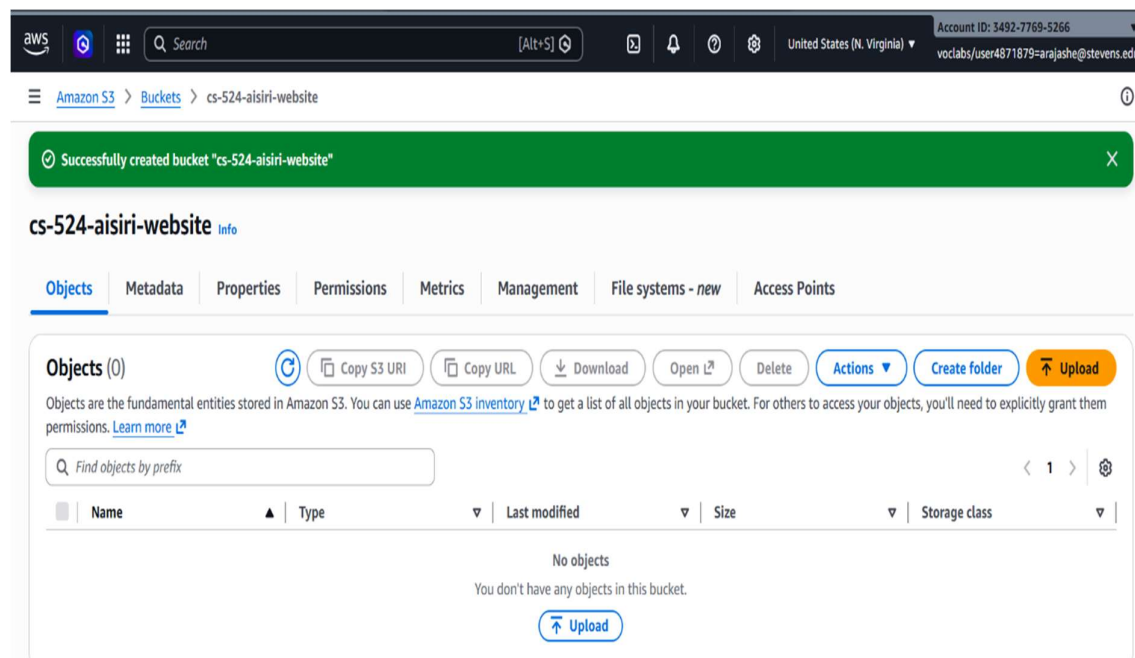
This architecture was designed to be secure, scalable, and cost-effective using managed AWS services. The static frontend (HTML, CSS, JavaScript) is hosted on Amazon S3 and delivered globally through CloudFront, which enforces HTTPS/TLS encryption for all client connections. This separation of frontend and backend allows the website to scale effortlessly without any server management.

When a user submits text for analysis, the browser sends a POST request directly to a Lambda Function URL. AWS Lambda runs the AI/NLP processing in a serverless environment, meaning it only consumes resources when invoked. Analysis results are stored as JSON files back in S3, creating a persistent record of each request. CloudWatch automatically captures Lambda execution logs, enabling monitoring, dashboards, and alarm-based notifications.

This architecture avoids the need for EC2 instances or traditional servers entirely, keeping costs within the AWS free tier for the expected workload while maintaining strong security through SSE-S3 encryption and HTTPS enforcement at every layer.

## Phase 2 — Create S3 Bucket & Host Static Website

### Step 1: Create the S3 Bucket



Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

Block public access to buckets and objects granted through new access control lists (ACLs)

S3 will block public access permissions except for newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

Block public access to buckets and objects granted through any access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

Block public access to buckets and objects granted through new public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

Block public and cross-account access to buckets and objects through any public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.

Turning off block all public access might result in this bucket and the objects within becoming public

AWS recommends that you turn on block all public access, unless public access is required for specific and verified use cases such as static website hosting.

☒ I acknowledge that the current settings might result in this bucket and the objects within becoming public.

Bucket Versioning

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

☒ Disable

☐ Enable

Tags - optional

You can use bucket tags to analyze, manage and specify permissions for a bucket. [Learn more](#)

You can use s3:ListTagForResources, s3:TagResource, and s3:UntagResource APIs to manage tags on S3 general purpose buckets for access control in addition to cost allocation and resource organization. To ensure a seamless transition, please provide permissions to s3:ListTagForResources, s3:TagResource, and s3:UntagResource actions. [Learn more](#)

No tags associated with this bucket.

[Add new tag](#)

You can add up to 50 tags.

Default encryption

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type

[Info](#)

☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)

☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)

☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

☐ Disable

☒ Enable

Advanced settings

After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

aws

Search

[Alt+S]

United States (N. Virginia)

Account ID: 3492-7769-5266

voclabs/user4871879-arajash@stevens.edu

Amazon S3

Buckets

Buckets

General purpose buckets

All AWS Regions

Directory buckets

General purpose buckets (1)

[Info](#)

[Copy ARN](#)

Empty

Delete

Create bucket

Buckets are containers for data stored in S3.

Find buckets by name

| Name                                  | AWS Region                      | Creation date                        |
|---------------------------------------|---------------------------------|--------------------------------------|
| <a href="#">cs-524-aisiri-website</a> | US East (N. Virginia) us-east-1 | April 27, 2026, 15:22:29 (UTC-04:00) |

Account snapshot

[Info](#)

[View dashboard](#)

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary

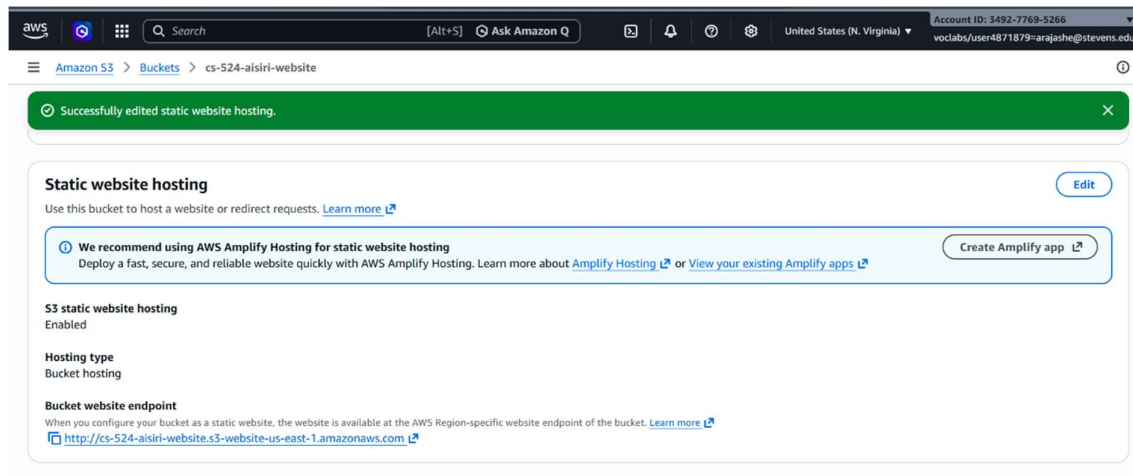
[Info](#)

Updated daily

External access summary helps you identify buckets

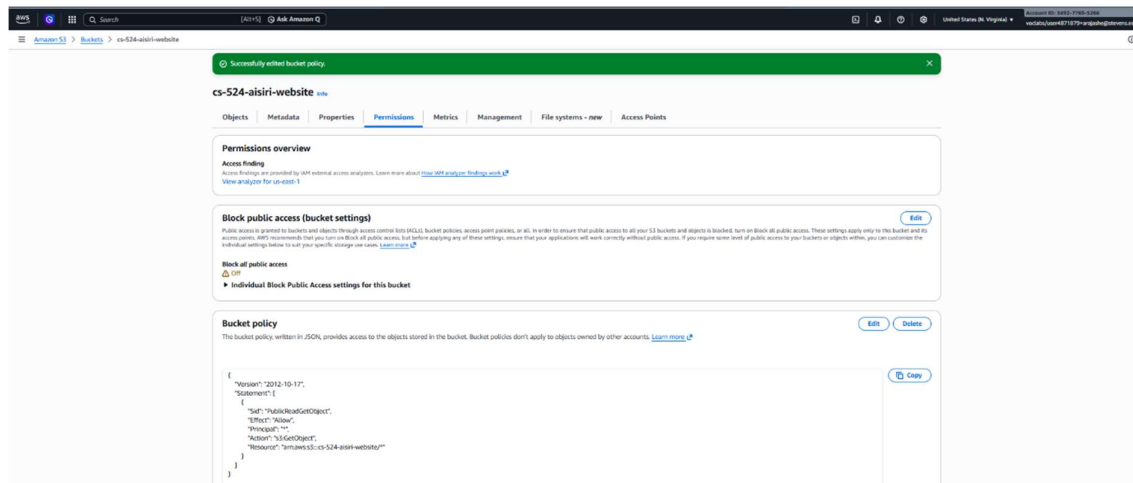
I created a new S3 bucket called cs-524-aisiri-website in the us-east-1 region. I unchecked Block all public access and acknowledged the warning before creating it.

## Phase 2: Static website hosting turned on, showing the bucket endpoint URL



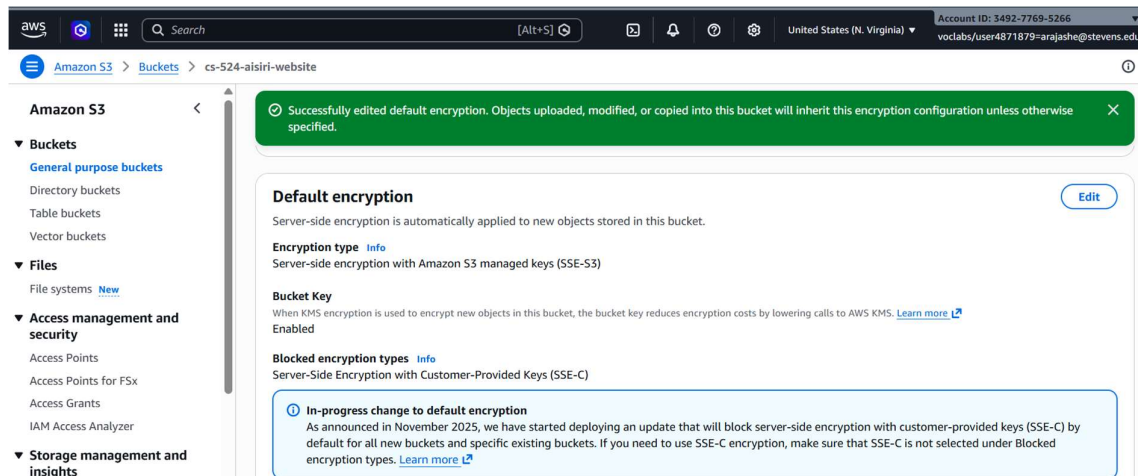
I went to the Properties tab and enabled static website hosting with index.html as the index document. The bucket endpoint URL appeared at the bottom which I saved for later.

### Step 3: Bucket policy added to allow public read access



I pasted the public read policy into the bucket policy editor under the Permissions tab. After saving, the bucket was accessible publicly for static hosting.

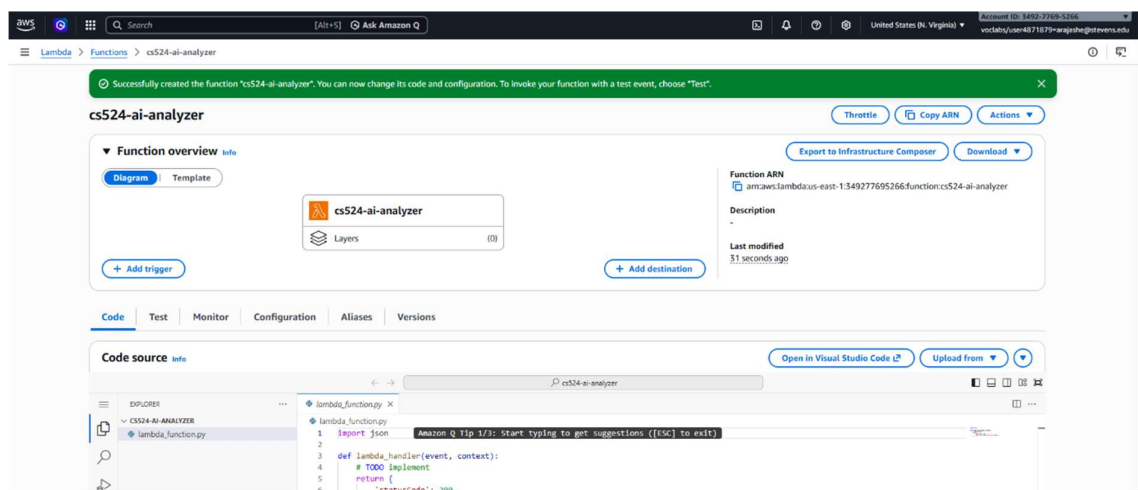
### Step 4: Default encryption set to SSE-S3



I went to Properties, found Default encryption, and changed it to SSE-S3. This makes sure any file stored in the bucket is encrypted automatically.

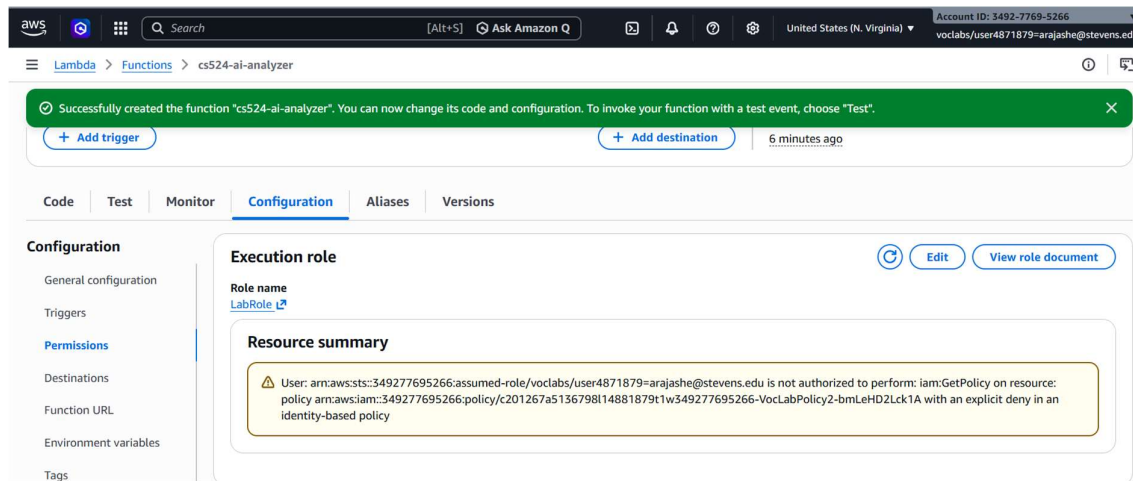
### Phase 3: Create Lambda Function (AI Backend)

#### Step 1a: Lambda function cs524-ai-analyzer successfully created



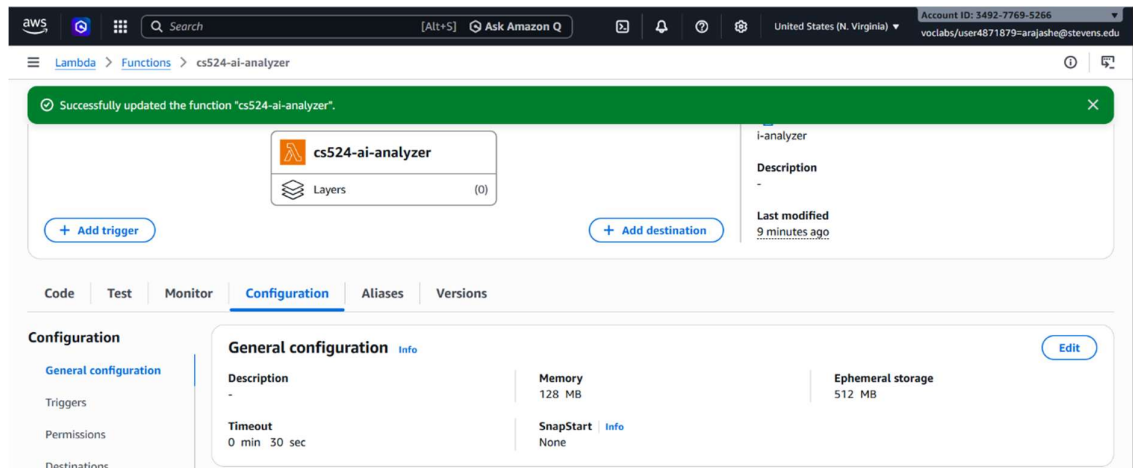
I created a new Lambda function from scratch using Python 3.12 and x86\_64 architecture. The green success banner confirmed it was created without any issues.

#### Step 1b: Permissions tab showing LabRole is attached as the execution role



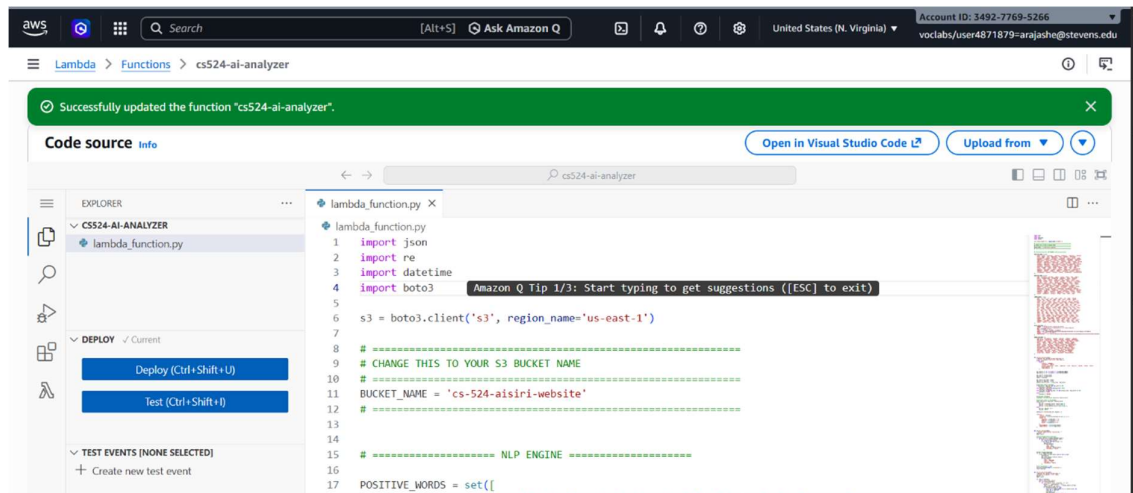
I went to the Configuration tab and clicked Permissions to verify the execution role. It showed LabRole was correctly attached as required.

## Step 2: Timeout updated to 30 seconds under general configuration



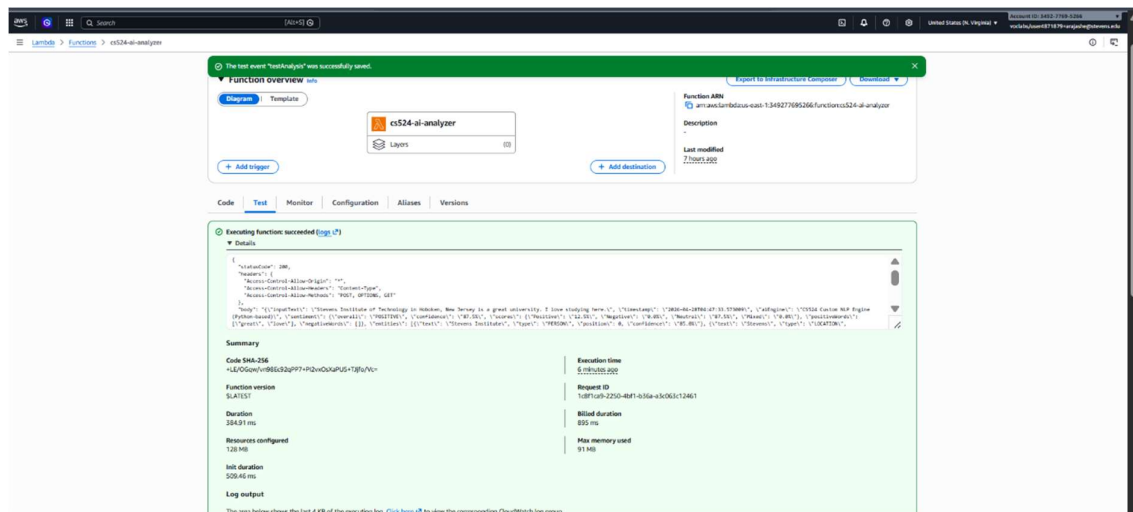
I went to Configuration, then General configuration, and clicked Edit. I changed the timeout from 3 seconds to 30 seconds and saved.

## Step 3: CA's Lambda code pasted and deployed successfully



I pasted the CA's Lambda code into the editor and updated the BUCKET\_NAME variable to match my S3 bucket. After clicking Deploy the green bar showed it was successfully updated.

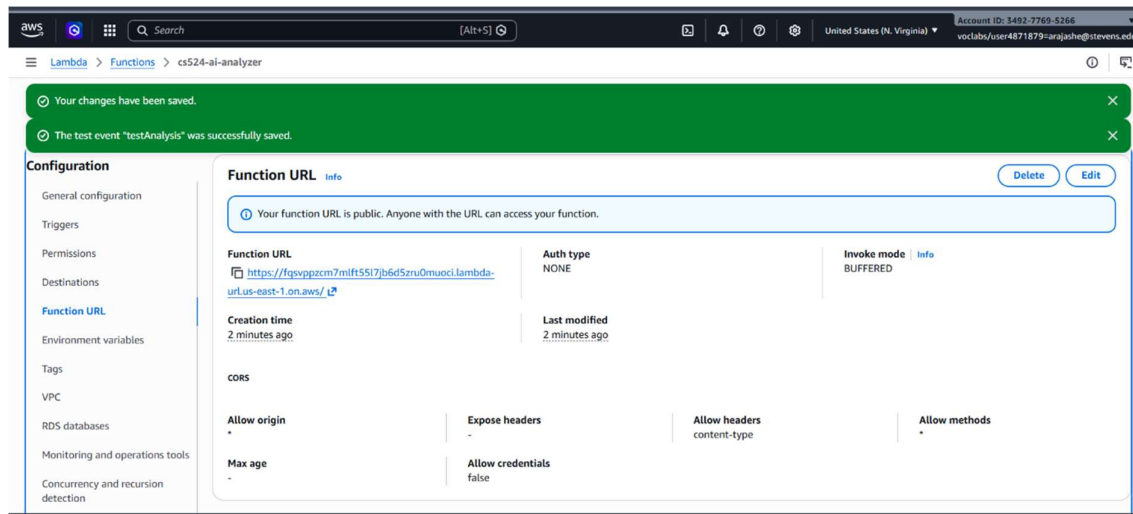
#### Step 4: Test result showing the function ran successfully with statusCode 200



I created a test event called testAnalysis with the sample JSON provided in the announcement. Running it returned statusCode 200 with full analysis results showing it was working.

#### Step 5: Function URL created with CORS settings showing allow origin set to star





I went to Configuration then Function URL and created a new URL with Auth type NONE. I enabled CORS with allow origin set to star and allow headers set to content-type.

## Phase 4: Upload Website & Connect to Lambda

### Step 1: script.js updated with the Lambda Function URL before uploading

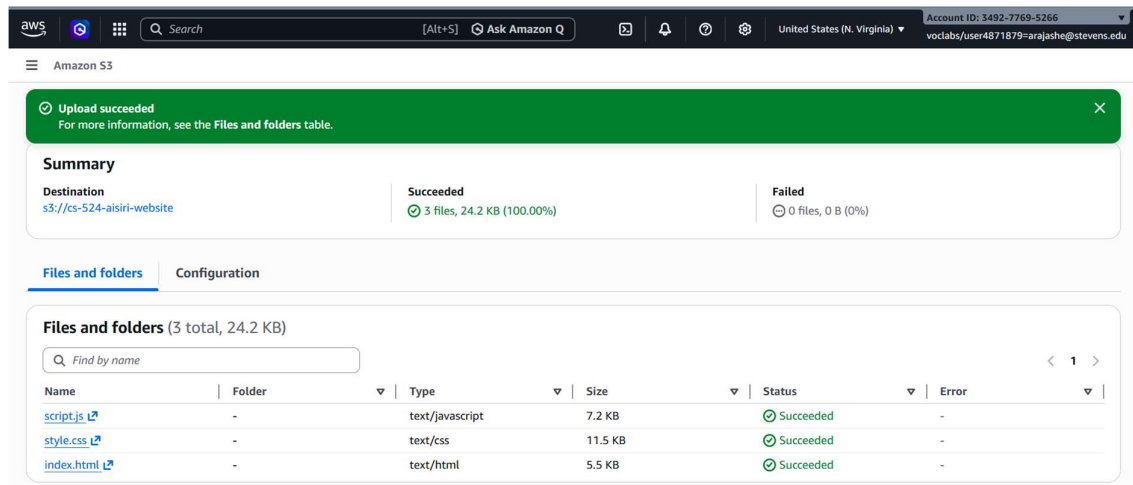
```

1 // =====
2 // CS524 - AI Text Analyzer
3 // Lambda Function URL - update this with your actual URL
4 // =====
5 const API_URL = 'https://fqsvppzcm7mlft55l7jb6d5zru0muoci.lambda-url.us-east-1.on.aws/';
6
7 // ===== Character Counter =====
8 const textInput = document.getElementById('textInput');
9 const charCount = document.getElementById('charCount');
10
11 textInput.addEventListener('input', () => {
12   const len = textInput.value.length;
13   charCount.textContent = `${len} character${len !== 1 ? 's' : ''}`;
14 });
15
16 // ===== Allow Ctrl+Enter to submit =====
17 textInput.addEventListener('keydown', (e) => {
18   if (e.key === 'Enter' && e.ctrlKey) analyzeText();
19 });
20
21 // ===== Main Analyze Function =====

```

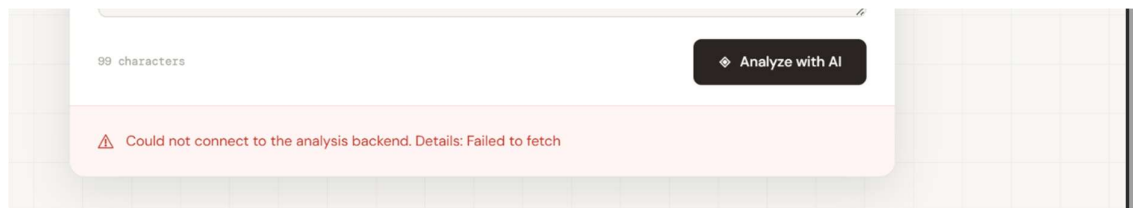
I opened script.js on my computer and replaced the placeholder text with my actual Lambda Function URL. I saved the file before uploading anything to S3.

### Step 2: All three files uploaded to the S3 bucket

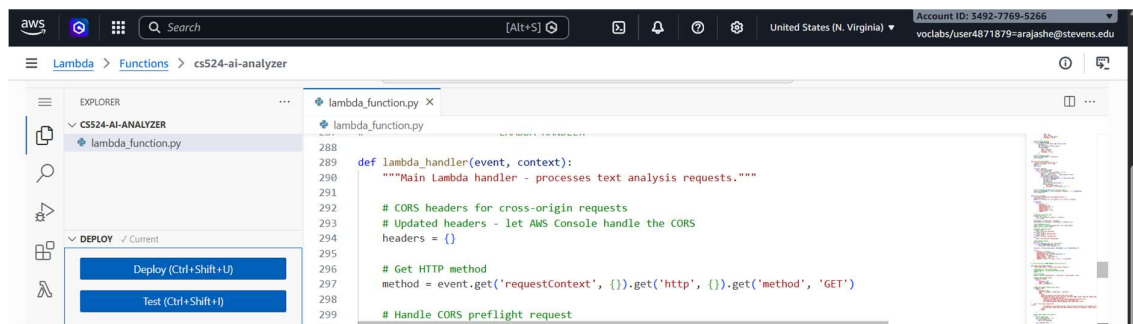


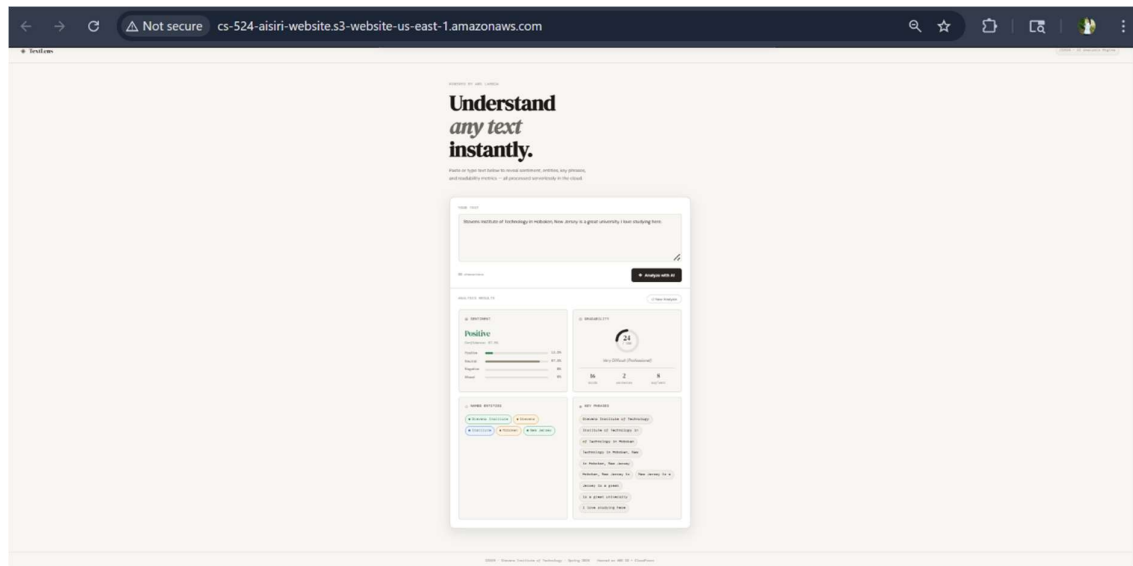
I went to my S3 bucket and uploaded all three files — index.html, style.css, and script.js. All three showed up in the bucket after the upload completed.

### Step 3: Website loading through the S3 endpoint URL



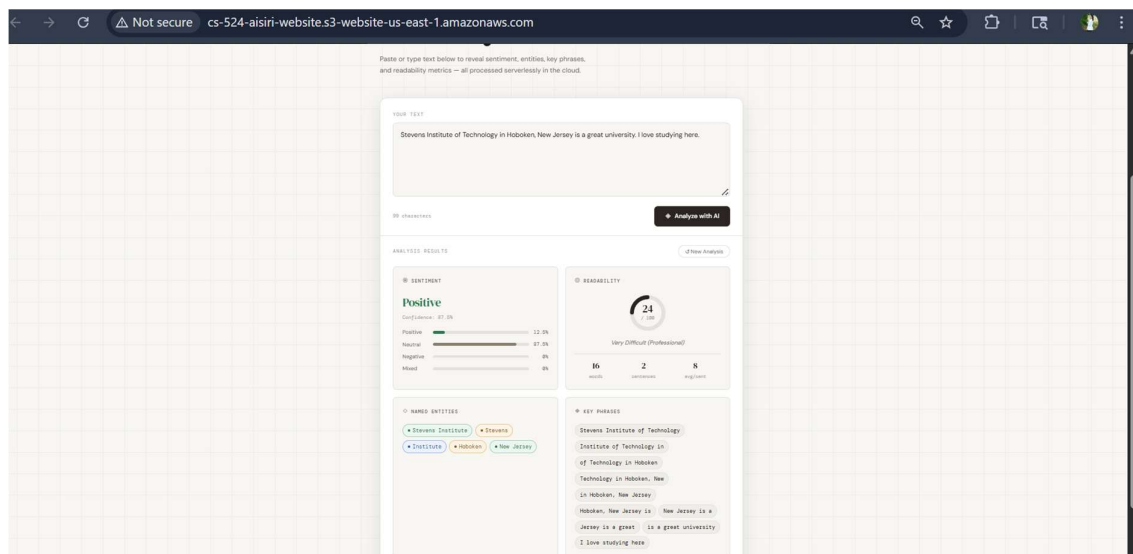
I made changes in the code because I was getting this error.

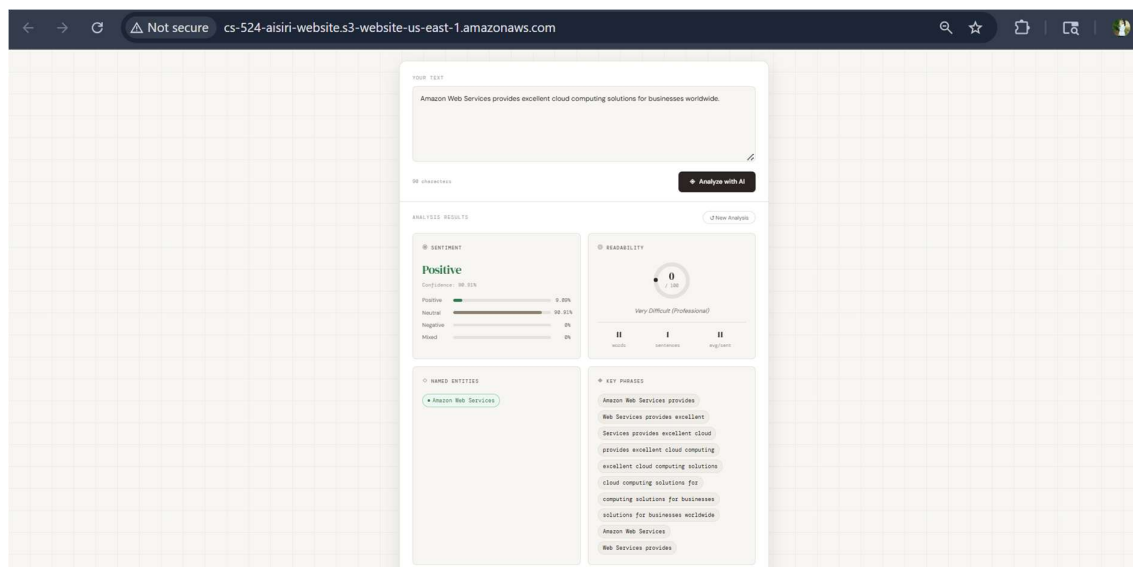
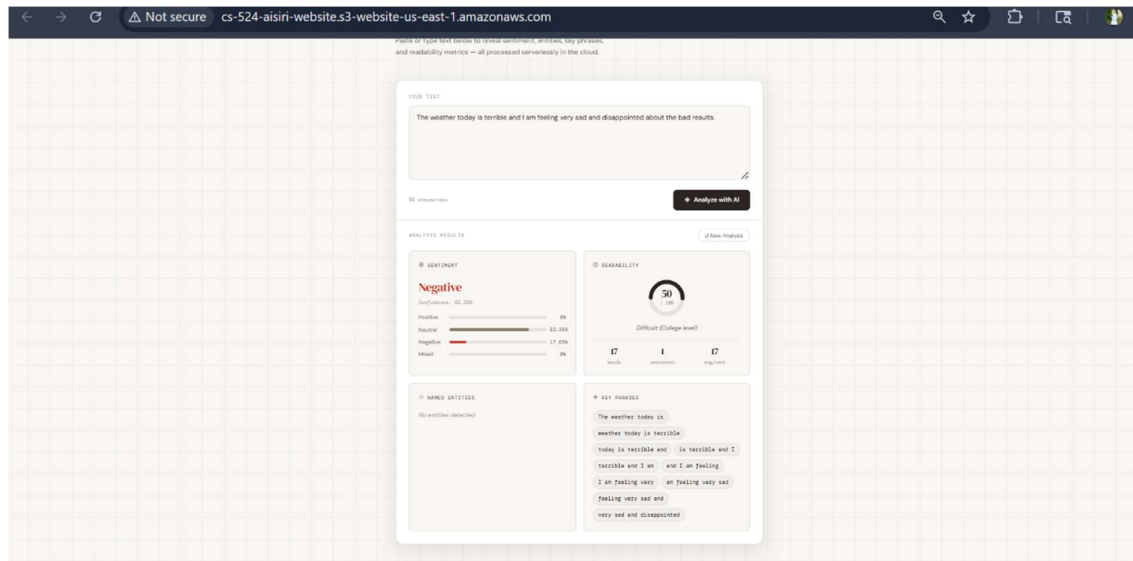




I copied the bucket website endpoint URL from the Properties tab and opened it in the browser. The website loaded correctly showing the text input and analyze button.

#### Step 4: Website showing the analysis results after entering sample text





I typed in the sample sentence and clicked Analyze with AI. The results section showed up with sentiment, entities, key phrases, and readability all displaying correctly.

### Step 5: Results folder in S3 showing JSON files saved from each analysis

**Amazon S3** > Buckets > cs-524-aisiri-webday > results/

**results/** [Copy S3 URI](#)

Objects (12)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

| Name                                          | Type | Last modified                         | Size   | Storage class |
|-----------------------------------------------|------|---------------------------------------|--------|---------------|
| <a href="#">analysis-20200428-047133.json</a> | json | April 28, 2020, 00:47:34<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-051119.json</a> | json | April 28, 2020, 02:11:41<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-051145.json</a> | json | April 28, 2020, 02:11:46<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-051746.json</a> | json | April 28, 2020, 02:19:47<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-158717.json</a> | json | April 28, 2020, 15:47:19<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-159344.json</a> | json | April 28, 2020, 15:53:05<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-159754.json</a> | json | April 28, 2020, 15:57:56<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-200551.json</a> | json | April 28, 2020, 16:02:52<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-200516.json</a> | json | April 28, 2020, 16:04:17<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-211031.json</a> | json | April 28, 2020, 17:10:32<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-211310.json</a> | json | April 28, 2020, 17:11:21<br>UTC-04:00 | 2.7 KB | Standard      |
| <a href="#">analysis-20200428-211755.json</a> | json | April 28, 2020, 17:17:55<br>UTC-04:00 | 2.7 KB | Standard      |

I went back to the S3 bucket and found a results folder had been created automatically. Inside it there were JSON files from each analysis I had run.

## Phase 5: CloudFront HTTPS Distribution

### Step 1: CloudFront distribution created showing the S3 origin settings

**Successfully created new distribution.** [View metrics](#)

**cs524-aisiri-distribution** [Enable](#)

**Details**

Distribution domain name: [d1568dmejp7y3s.cloudfront.net](#)

Billing: [Unable to determine billing status.](#)

ARN: [arn:aws:cloudfront:349277606266:distribution/E1051HALPND0H6](#)

Last modified: April 28, 2020 at 5:21:41 AM UTC

**Everything you need for a simple monthly price**

- Plans include global CDN, WAF, DDoS protection, DNS, TLS certificate, log inspection, and serverless edge compute.
- No overage charges.
- Blocked requests and DDoS attacks never count against your usage allowances.
- Data transfer costs from your AWS origins (such as Amazon S3, Application Load Balancer, or API Gateway) to CloudFront are automatically waived.

[Learn more](#)

**General** | Security | Origins | Behaviors | Error pages | Invalidations | Logging | Tags

**Settings** [Edit](#)

Name: cs524-aisiri-distribution

Description: -

Price class: One all-edge locations (best performance)

Supported HTTP versions: HTTP/2, HTTP/1.1, HTTP/1.0

Alternate domain names: [Add domain](#)

Standard logging: [Off](#)

Cookie logging: [Off](#)

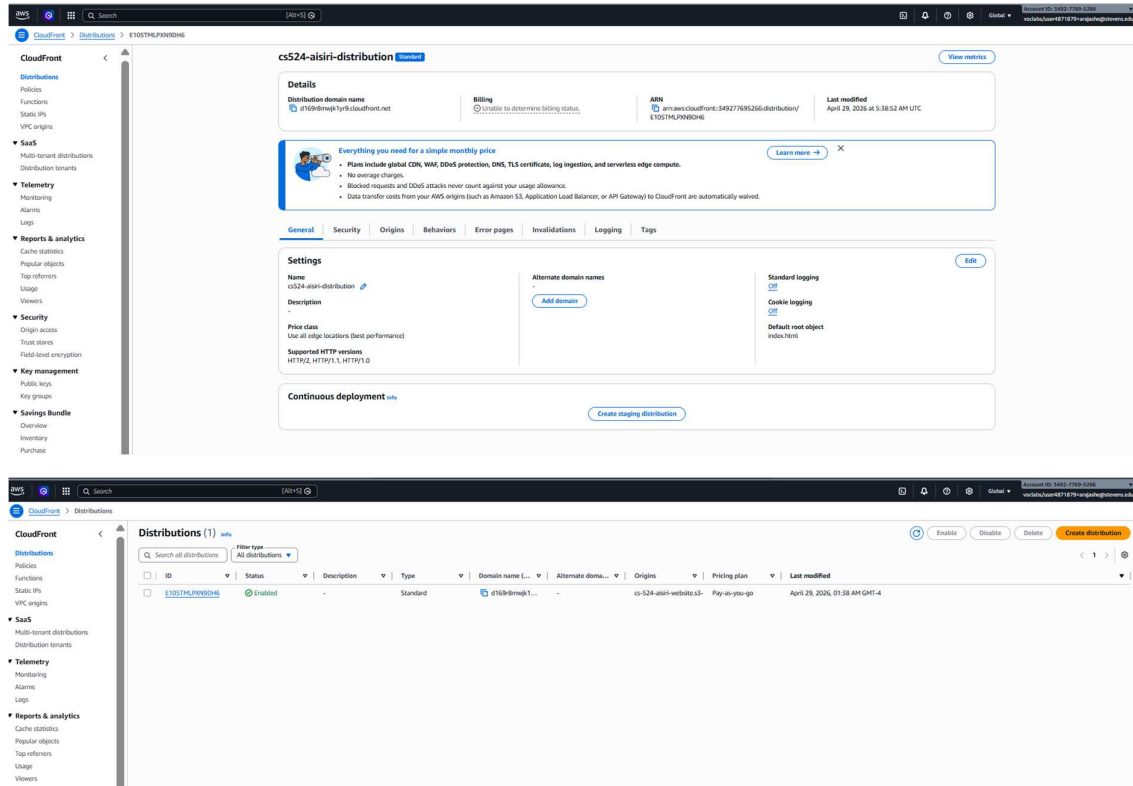
Default root object: -

**Continuous deployment** [View](#)

[Create staging distribution](#)

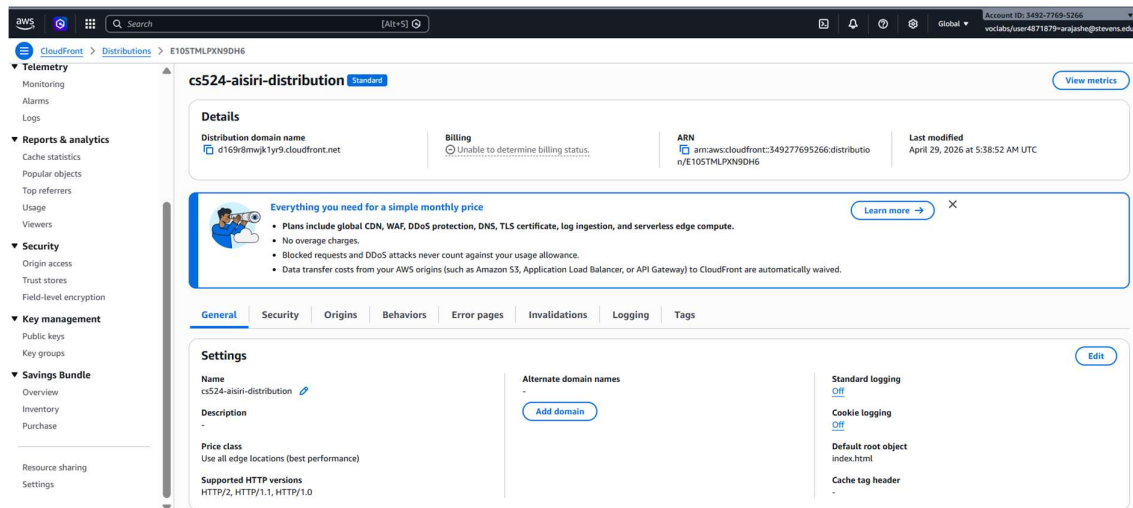
I opened CloudFront and created a new distribution using the Pay-as-you-go plan. I selected my S3 bucket as the origin and clicked Use website endpoint when the yellow banner appeared.

### Step 2: Distribution status showing Enabled



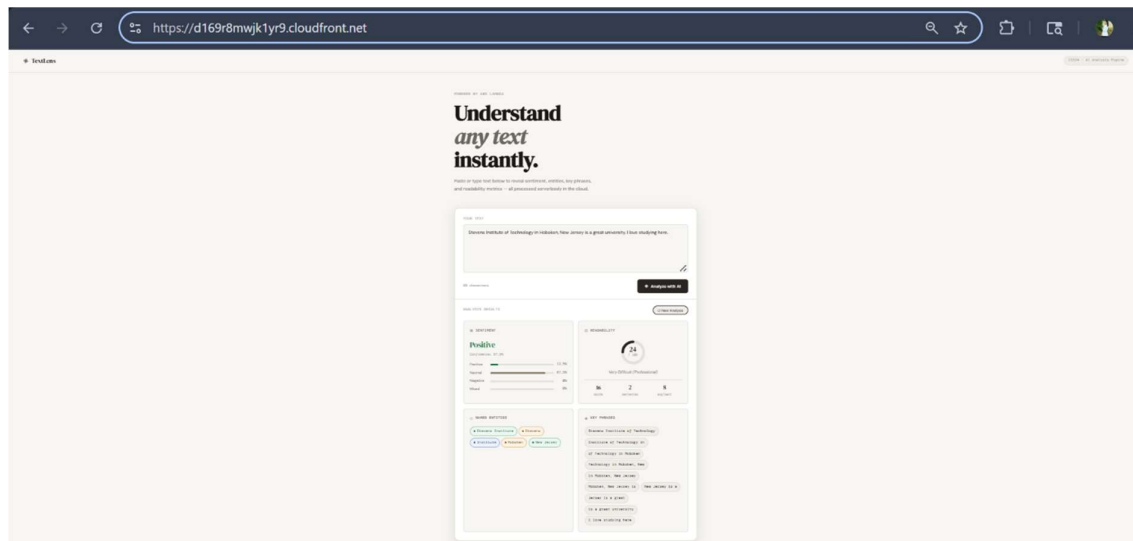
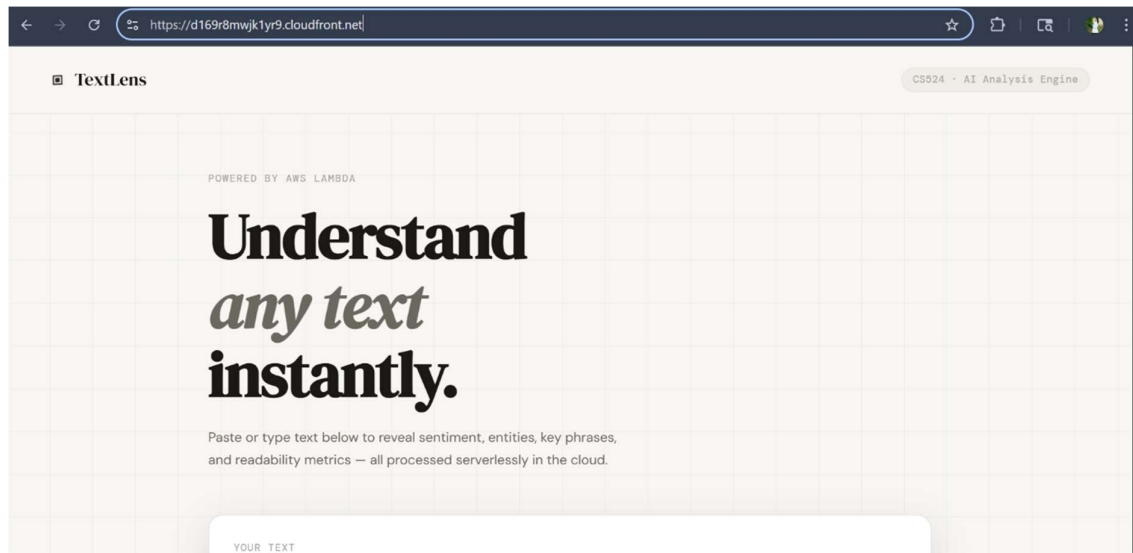
After creating the distribution I waited about 10 minutes refreshing the page. The status eventually changed from Deploying to Enabled.

### Step 3: Default root object set to index.html in general settings



I clicked Edit in the General settings of the distribution and typed index.html as the default root object. This makes sure the homepage loads when someone visits the CloudFront URL.

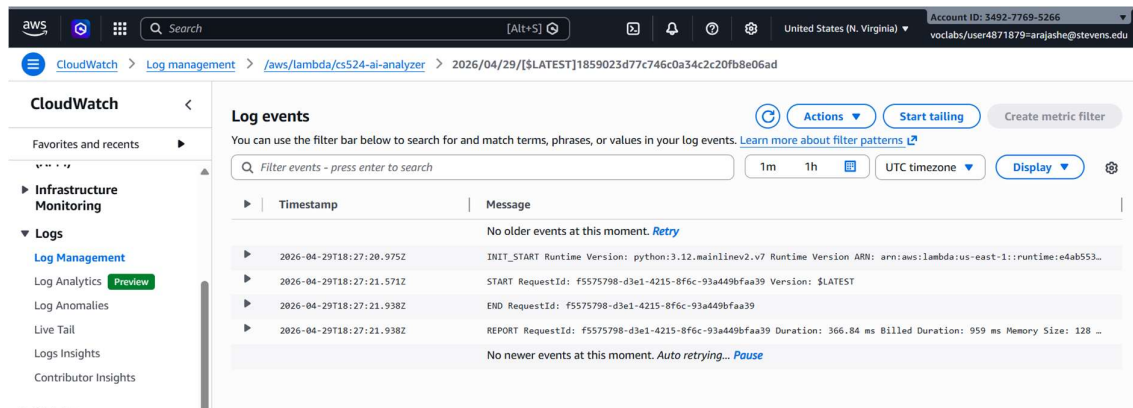
### Step 4: Website loading over HTTPS using the CloudFront URL



I copied the CloudFront domain name and opened it in the browser with https. The website loaded and the Analyze button worked correctly this time without any errors.

## Phase 6: CloudWatch Monitoring

### Step 1: CloudWatch log group for the Lambda function showing execution logs

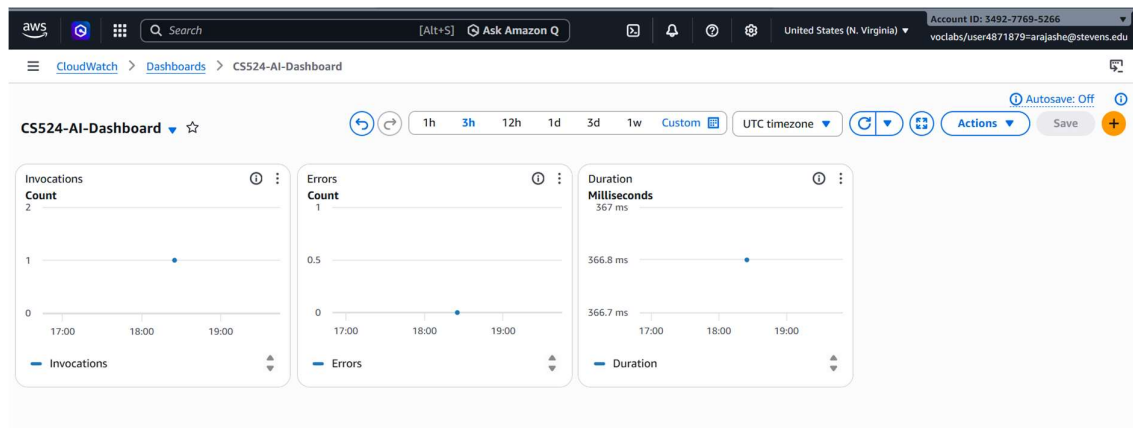


The top screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation links for CloudWatch, Log management, and various monitoring tools. The main content area displays 'Log events' for a specific Lambda function. A search bar is present, and a table of log entries is shown with columns for 'Timestamp' and 'Message'. The messages include details about function runtime, request IDs, and response times.

The bottom screenshot shows the same CloudWatch console, but with a different set of log entries. These entries provide more detailed information, including request IDs, response times, and memory usage. The interface includes filters for time range (1m, 1h, 30m, 1h, 12h, Custom) and a 'Display' button.

I opened CloudWatch and went to Log groups where I found the log group for my Lambda function. I clicked into one of the log streams and could see entries from each time the function ran.

## Step 2: CS524-AI-Dashboard with Invocations, Errors and Duration widgets.



I created a new dashboard called CS524-AI-Dashboard and added three widgets for Invocations, Errors, and Duration. After running a few more analyses the widgets started showing real data.

## Step 3: CloudWatch alarm created for Lambda errors



**CloudWatch** > Alarms > cs524-lambda-errors

**Alarms (1)**

Filter alarms

Hide Auto Scaling alarms

< 1 >

**cs524-lambda-errors**

Metric alarm

Insufficient data

**Details**

|                                                       |                                                                  |                                                                                   |                                                                |
|-------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Name</b><br>cs524-lambda-errors                    | <b>State</b><br>Insufficient data                                | <b>Namespace</b><br>AWS/Lambda                                                    | <b>Datapoints to alarm</b><br>1 out of 1                       |
| <b>Type</b><br>Metric alarm                           | <b>Threshold</b><br>Errors > 5 for 1 datapoints within 5 minutes | <b>Metric name</b><br>Errors                                                      | <b>Missing data treatment</b><br>Treat missing data as missing |
| <b>Description</b><br>No description                  | <b>Actions</b><br>Actions enabled                                | <b>FunctionName</b><br>cs524-ai-analyzer                                          | <b>Percentiles with low samples</b><br>evaluate                |
| <b>Last state update</b><br>2026-04-29 20:44:25 (UTC) | <b>Statistic</b><br>Sum                                          | <b>ARN</b><br>arn:aws:cloudwatch:us-east-1:349277695266:alarm:cs524-lambda-errors |                                                                |
|                                                       | <b>Period</b><br>5 minutes                                       |                                                                                   |                                                                |

**CloudWatch** > Alarms

Successfully created alarm cs524-lambda-errors. View alarm

Some subscriptions are pending confirmation. Amazon SNS doesn't send messages to an endpoint until the subscription is confirmed. View SNS Subscriptions

**Alarms** | Mute rules - new

**Alarms (1)**

Filter alarms

Quick filters

Clear selection

Create composite alarm

Actions

Create alarm

| Name                | State             | Actions         | Actions muted - new | Last state update (UTC) | Conditions                                   |
|---------------------|-------------------|-----------------|---------------------|-------------------------|----------------------------------------------|
| cs524-lambda-errors | Insufficient data | Actions enabled | -                   | 2026-04-29 20:02:52     | Errors > 5 for 1 datapoints within 5 minutes |

**CloudWatch** > Alarms

**Alarms** | Mute rules - new

**Alarms (1)**

Filter alarms

Quick filters

Clear selection

Create composite alarm

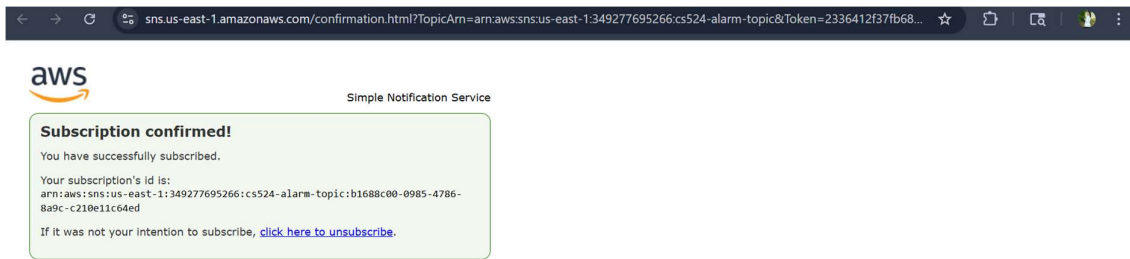
Actions

Create alarm

| Name                | State             | Actions         | Actions muted - new | Last state update (UTC) |
|---------------------|-------------------|-----------------|---------------------|-------------------------|
| cs524-lambda-errors | Insufficient data | Actions enabled | -                   | 2026-04-29 20:44:25     |

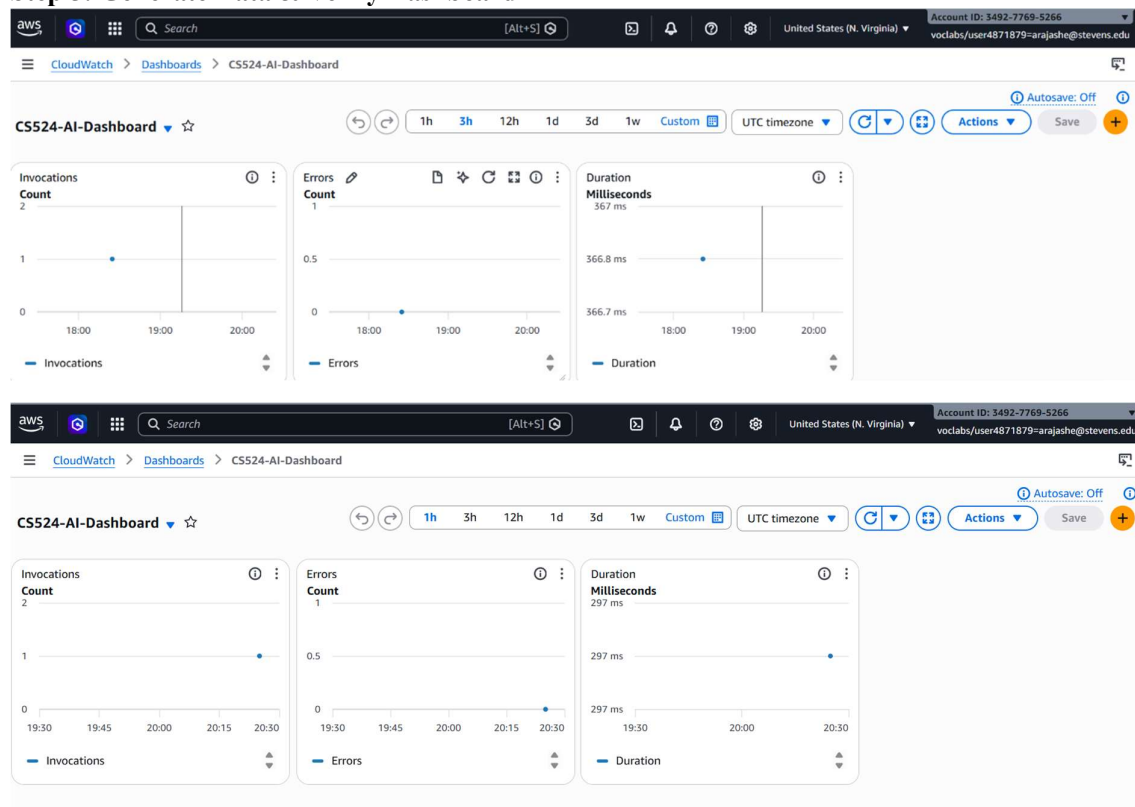
I created a CloudWatch alarm based on the Lambda Errors metric with a threshold of 5 errors in 5 minutes. I set up an SNS topic with my email address to receive notifications if the alarm triggers.

#### Step 4: SNS subscription confirmation email received and confirmed



I checked my Stevens email and found the AWS subscription confirmation message. I clicked the confirm link in the email and the SNS subscription status updated to confirmed.

### Step 5: Generate Data & Verify Dashboard



I ran a few text analyses on the website to generate some actual usage data. Then I went back to the CloudWatch dashboard and the widgets updated showing real numbers for invocations and duration.

